

Reporte QA — Flujo de Consulta (Consultation Full Flow)

Fecha: 24 de junio de 2026 | Tester: Pedro Quijada | Entorno: admin-dev.mediplanner.mx

Alcance

4 corridas del flujo completo de consulta (cita → signos vitales → General → Exploración → Diagnóstico → Tratamiento → Notas → Servicios → finalizar), con monitor de DevTools capturando llamadas API y errores de consola.

Hallazgo principal: 500 intermitente al guardar Exploración

POST /api/patients/registerAnswers → **500 "Server Error"** al guardar los apartados de Exploración. Reproducido en 1 de 4 corridas (intermitente).

Capa	Problema
Front	Ante un solo "Guardar", envía el request dos veces en 40 ms (doble submit).
Backend	Falla con 500 cuando los dos requests concurrentes chocan (race condition: sin idempotencia / manejo de concurrencia).
Front	No maneja el 500 → crash JS Cannot read properties of undefined (reading 'data').

Evidencia (traza de red):

```
+82.12s REQUEST registerAnswers 2 requests por 1 click
+82.16s REQUEST registerAnswers (el test hace 1 click cada ~2s)
+82.75s RESPONSE 500 Server Error
+82.76s JS ERROR Cannot read properties of undefined (reading 'data')
+83.06s RESPONSE 200 OK el reintento salva el dato
```

Hallazgo relacionado: mismo patrón en Tratamiento

POST /api/consultations/setTreatments → **500** observado en sesión previa (con Atorvastatina), con idéntico patrón de requests duplicados/concurrentes. **No es un endpoint específico** — es un problema transversal del backend ante escrituras paralelas.

Lo que está sano

- ✓ Flujo completo de consulta: pasa de punta a punta en las 4 corridas.
- ✓ Indicador "sin guardar" (triángulo): corregido, limpio en todos los apartados.
- ✓ 0 errores reales de consola fuera de los 500 (el resto es ruido externo: GA / Zendesk / Clarity).

Conclusión

La plataforma **funciona pero es frágil bajo escrituras concurrentes**. El doble-submit del front más la falta de manejo de concurrencia en el backend producen un **500 intermitente y auto-recuperable** (el reintento salva el dato, por eso el usuario rara vez lo nota).

Severidad media: no hay pérdida de datos confirmada, pero el crash JS asociado y la dependencia del reintento lo hacen un riesgo real.

Recomendación: corregir el doble-submit en el front (raíz más barata) y dar idempotencia / manejo de concurrencia a `registerAnswers` y `setTreatments` en el backend.

Generado a partir de 4 corridas automatizadas con Playwright + monitor DevTools.